

SMART NFC ATTENDANCE SYSTEM

ESP32 + RC522 + Python + SQLite

Project README & Complete Setup Guide

A student-built engineering project for automated NFC/RFID attendance.

1. Project Overview

The Smart NFC Attendance System is an end-to-end attendance platform that uses an RC522 RFID/NFC reader to identify a supported card or tag, an ESP32 microcontroller to read its UID, and a Python desktop application to register students, record attendance, search records, and export attendance data to CSV.

Core data flow:

NFC/RFID Card → RC522 → ESP32 → USB Serial → Python → SQLite → GUI / CSV

The project combines electronics, embedded programming, serial communication, databases, GUI development, and real-world system design.

2. Main Features

- NFC/RFID card UID detection
- ESP32-based reader controller
- USB serial communication with a laptop
- Student registration
- Attendance marking by UID
- Duplicate-attendance prevention for the same day
- Attendance search
- CSV export
- Desktop GUI with Scanner, Students, Attendance, and ESP32 sections
- SQLite database for persistent records

3. Hardware Requirements

Component	Purpose
ESP32 NodeMCU / ESP32-WROOM-32 with CP2102	Microcontroller and USB-to-serial connection
RC522 RFID/NFC reader	Reads supported 13.56 MHz cards/tags
NFC/RFID card or tag	Unique UID used for attendance
Breadboard	Temporary circuit assembly
Jumper wires	Electrical connections
USB data cable	ESP32 power + programming + serial communication
Windows PC/Laptop	Arduino IDE + Python attendance application

4. RC522 → ESP32 Wiring

RC522 Pin	ESP32 Pin	Notes
3.3V	3V3	Use 3.3 V only
GND	GND	Common ground
SDA / SS	GPIO 5	SPI chip select
SCK	GPIO 18	SPI clock
MOSI	GPIO 23	SPI data to RC522
MISO	GPIO 19	SPI data from RC522
RST	GPIO 22	Reset
IRQ	Not connected	Not required

Important: The RC522 is a 3.3 V device. Do not power the RC522 from 5 V.

5. Install Arduino IDE on Windows

1. Download Arduino IDE from the official Arduino website.
2. Install it using the normal Windows installer.
3. Accept the default installation options unless you have a specific reason to change them.
4. Open Arduino IDE after installation.

6. Add ESP32 Board Support

Open **File** → **Preferences**. Find **Additional boards manager URLs** and add the official Espressif package URL:

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

Click OK. Then open **Tools** → **Board** → **Boards Manager**, search for **esp32**, and install **esp32 by Espressif Systems**.

7. Install the MFRC522 Library

Open **Tools** → **Manage Libraries**. Search for **MFRC522** and install the commonly used MFRC522 library by the Arduino MFRC522 community/miguelbalboa project.

8. Connect and Select the CP2102 ESP32

Connect the ESP32 to the PC with a USB **data** cable. In Arduino IDE open **Tools** → **Port** and identify the new COM port. If unsure, unplug the ESP32, check the available ports, reconnect it, and identify the newly appearing COM port.

For a common ESP32-WROOM-32 / NodeMCU-style board, select **Tools** → **Board** → **ESP32 Arduino** → **ESP32 Dev Module**. The CP2102 is the USB-to-serial bridge; it does not mean you should select a board called “CP2102.”

9. Open the Project Sketch

Open the supplied **ESP32_NFC_Attendance.ino** file in Arduino IDE. Check that the board and COM port are still selected correctly.

10. Verify / Compile

Click the checkmark **Verify** button. Arduino IDE will compile the sketch. If compilation fails, copy the complete error message or take a screenshot before changing settings.

11. Upload to the ESP32

Click the right-arrow **Upload** button. The IDE will compile and then transfer the program to the ESP32.

If the IDE remains stuck at **Connecting...**, press and hold the board's **BOOT** button when the upload begins. Release it once writing starts. Some ESP32 boards require this manual bootloader step.

12. Test the Serial Output

After uploading, open **Tools** → **Serial Monitor** and set the baud rate to **115200**. The sketch should display its startup information and indicate that it is waiting for a card.

13. Python Application

The desktop application uses Python with Tkinter for the GUI, SQLite for storage, CSV for export, and pyserial for USB serial communication.

```
Python GUI
↓
SQLite database: attendance.db
↓
Students + Attendance records
```

The application supports student registration, manual UID testing, attendance search, CSV export, and ESP32 COM-port connection. The ESP32 listener expects card UID information through the serial connection.

14. Python Installation

Install Python 3 for Windows. Verify it from a terminal with:

```
python --version
```

If pip is not available as a standalone command, use Python's module form:

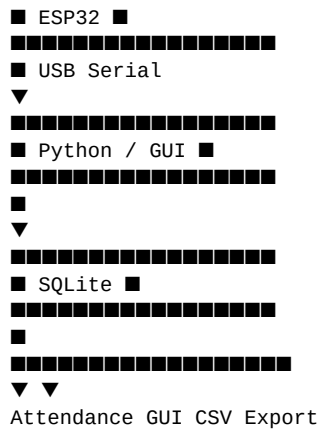
```
python -m pip install pyserial
```

Tkinter and SQLite are normally included with standard Python installations on Windows.

15. Project Folder Structure

```
NFC-Attendance-System/
■■■ ESP32_NFC_Attendance.ino
■■■ main.py
■■■ attendance.db # created by the Python app
■■■ attendance.csv # optional exported attendance
■■■ README.md
```

16. Software Architecture



17. Attendance Database Design

The system uses two main logical tables:

Students: stores UID, name, roll number, class, and division.

Attendance: stores the student/UID, date, and attendance timestamp or related attendance information.

A daily duplicate check prevents the same student from being marked present repeatedly on the same date.

18. Testing Plan

Test	Expected result
ESP32 connects to PC	A COM port appears
Sketch compiles	No compile errors
Sketch uploads	Upload completes successfully
Serial monitor at 115200	Startup messages appear
RC522 powered at 3.3 V	Reader initializes
Card scan	UID is detected
Student registration	Student is stored in SQLite
Attendance scan	Attendance record is created
Second scan same day	Duplicate is prevented
Search	Matching records are shown
CSV export	Attendance data is exported

19. Troubleshooting

Port does not appear: Try another USB data cable/USB port and check Windows Device Manager. The CP2102 driver may need to be installed depending on the board/Windows setup.

Upload stuck on Connecting: Confirm the correct COM port and board, then try holding BOOT during the connection stage.

MFRC522 compile error: Confirm that the MFRC522 library is installed and that the correct library is being

included.

RC522 not detected: Recheck 3.3 V, GND, SPI pins, SDA/SS, and RST. Avoid powering the module from 5 V.

Python cannot connect: Close Arduino Serial Monitor before the Python app opens the same COM port. Only one application can normally own the serial port at a time.

Wrong attendance values: The ESP32 serial protocol and Python parser must agree on exactly what constitutes a UID.

20. Important Serial Integration Note

The original ESP32 sketch contains human-readable debug messages as well as a UID line. The Python listener should not treat every serial line as an attendance UID. For the final integrated version, use a machine-readable format such as **UID:A3 7F 21 9C** and make Python process only lines beginning with **UID:.** This prevents messages such as “Card detected!” from becoming accidental attendance entries.

```
ESP32 → UID:A3 7F 21 9C
Python → accepts only lines beginning with UID:
```

21. Security / Reliability Considerations

For a school prototype, a card UID can be used as an identifier. For a production attendance system, UID-only identification should not be considered strong authentication because some RFID technologies may permit identifiers to be copied or emulated. A production design could add stronger credentials, server-side validation, access controls, audit logs, and encrypted communication.

22. Future Improvements

- Wi-Fi-based attendance synchronization
- Web dashboard for teachers/admins
- Offline attendance queue with later synchronization
- Custom PCB instead of a breadboard
- 3D-printed protective enclosure
- OLED/LCD status display
- Buzzer/LED feedback
- Real-time attendance statistics
- User authentication for administrators
- Cloud database integration
- Better card/tag validation
- Hardware power management
- Automated testing and logging

23. Recommended Portfolio Documentation

To make the project useful as an engineering portfolio piece, document the engineering process rather than only showing the finished prototype. Include:

1. Problem statement
2. Design requirements
3. Block diagram
4. Circuit schematic and wiring
5. Component selection and justification
6. ESP32 firmware

7. Python application
8. Database design
9. Testing results
10. Problems encountered and how they were solved
11. Iterations and improvements
12. Final prototype photos
13. Short demonstration video
14. GitHub repository with a clean README

24. Suggested V1 → V5 Development Path

V1 — Basic reader: RC522 + ESP32 reads UID.

V2 — Computer integration: ESP32 sends UID to Python over USB serial.

V3 — Complete attendance system: Student registration, SQLite, duplicate prevention, search and CSV export.

V4 — Connected system: Wi-Fi and web dashboard, with offline storage/synchronization.

V5 — Engineering prototype: Custom PCB, enclosure, polished UI, validation, documentation and reliability testing.

25. Quick Start Checklist

- Install Arduino IDE
- Add Espressif ESP32 Boards Manager URL
- Install “esp32 by Espressif Systems”
- Install MFRC522 library
- Connect ESP32 by USB data cable
- Select ESP32 Dev Module
- Select the correct COM port
- Open ESP32_NFC_Attendance.ino
- Verify/compile
- Upload
- If needed, hold BOOT while connecting
- Open Serial Monitor at 115200
- Wire RC522 to the ESP32
- Test a card/tag
- Connect Python application
- Register the card UID to a student
- Test attendance and duplicate prevention
- Export a CSV

26. Engineering Summary

This project demonstrates a complete hardware-to-software pipeline: a physical identification device is interfaced with a microcontroller, the microcontroller communicates with a computer, and application software converts the raw UID into a useful attendance record. The strongest version of the project is one where the build process, testing, failures, measurements, design decisions, and improvements are documented alongside the final working system.

README prepared for the Smart NFC Attendance System